ASSIGNMENT · RAG DATA INGESTION

Ingesting User Stories into MongoDB Atlas for RAG

An Excel-to-vector pipeline: convert 190 Health Care user stories to JSON, embed them with Mistral AI, and store them in MongoDB Atlas for vector search.

Node.js / Express | MongoDB Atlas | Mistral `mistral-embed` | Postman | VS Code

Summary

The pipeline ran end to end without errors. All 190 user stories were converted, embedded and stored in the `Janani_Collection2` collection in MongoDB Atlas.

190	100%	30,678	$0.0031
User stories ingested	Embedding success rate (0 failed)	Tokens embedded (avg 161 per story)	Total embedding cost

Pipeline

Excel userstories.xlsx sheet "stories" → **Upload API** POST /api/upload-excel → **JSON** stories-*.json 190 records → **Embeddings** Mistral mistral-embed → **MongoDB Atlas** Janani_Collection2 + vector index

Configuration

SETTING	VALUE	PURPOSE
Database	`JananiKhartik`	Atlas database on Cluster0
Test case collection	`Janani_Collection`	Existing test case data (vector + BM25 indexes)
User story collection	`Janani_Collection2`	Target for this assignment
Vector index	`vector_index_user_stories`	Semantic search over story embeddings
Embedding model	`mistral-embed`	Generates embedding vectors (Mistral AI)
API server	`http://localhost:3001/api`	Express backend driven from Postman

Stored document schema

Each user story is stored as one document with Jira-style fields:

FIELD	EXAMPLE (HC-257)
`key`	HC-257
`summary`	Consultant Dashboard My Inpatients
`description`	"As a user, I want to consultant dashboard my inpatients so that the related functionality works as expected."
`status`, `priority`	Nested objects
`project` / `epic`	Health Care / Emergency Management
`created` / `updated`	ISO timestamps
`components`, `labels`, `fixVersions`	Arrays (empty in this dataset)

Implementation steps

Each step below includes the screenshot that shows it working.

1 Configure the environment

The `.env` file sets the Atlas connection string, database and collection names, vector index names and the Mistral embedding model. User stories go to a separate collection from test cases, so the two datasets stay independent.

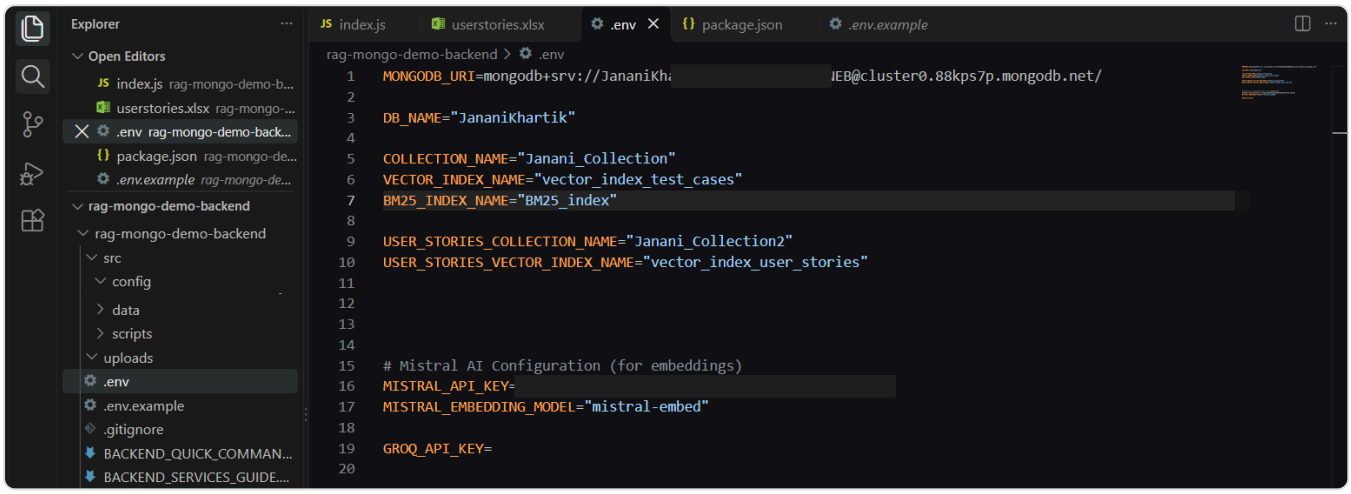


Figure 1. `.env` configuration in VS Code. The database password and Mistral API key are redacted.

2 Start the backend server

`npm run dev` starts the Express server on port 3001 and exposes the API at `/api`.

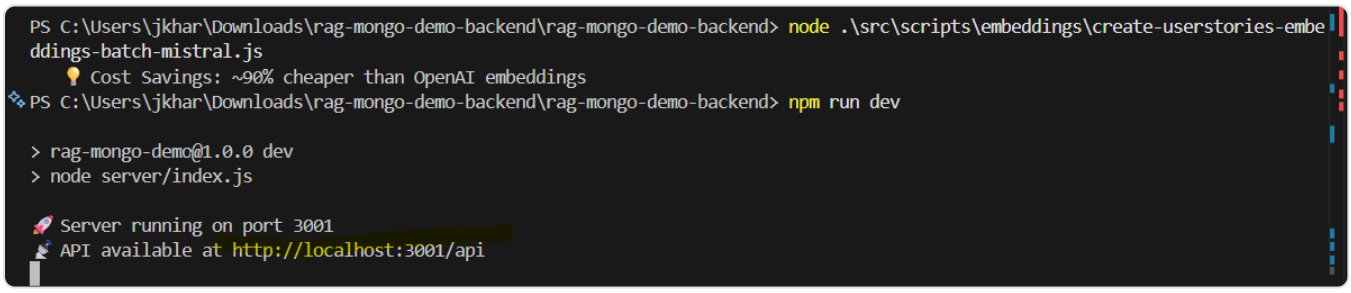


Figure 2. Server running on port 3001.

3 Verify the API with a health check

A `GET {{baseUrl}}/api/health` request from the Postman collection *RAG MongoDB – Data Ingestion* confirms the server is reachable.

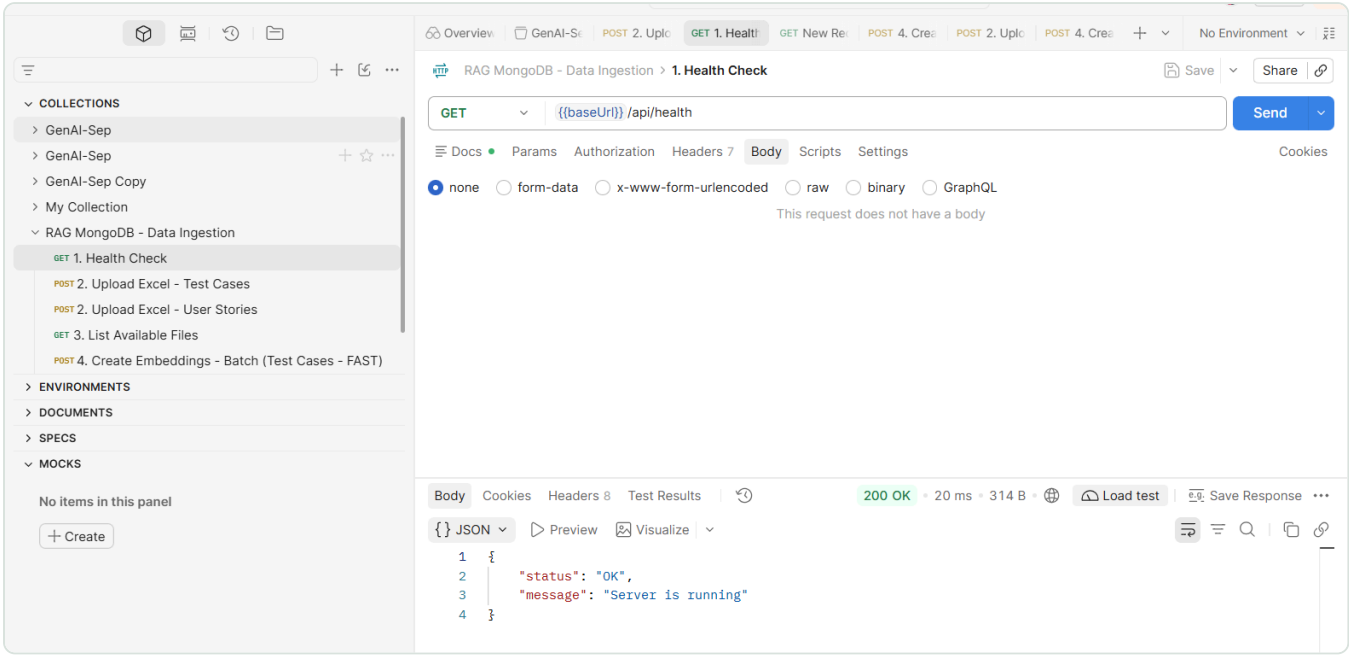


Figure 3. Health check returns `200 OK` in 20 ms with `{"status":"OK","message":"Server is running"}`.

4 Upload and convert the Excel file

`POST {{baseUrl}}/api/upload-excel` is sent as form-data with three fields: `file` = userstories.xlsx, `dataType` = userstories and `sheetName` = stories. The server converts every row to JSON and writes it to `src/data`.

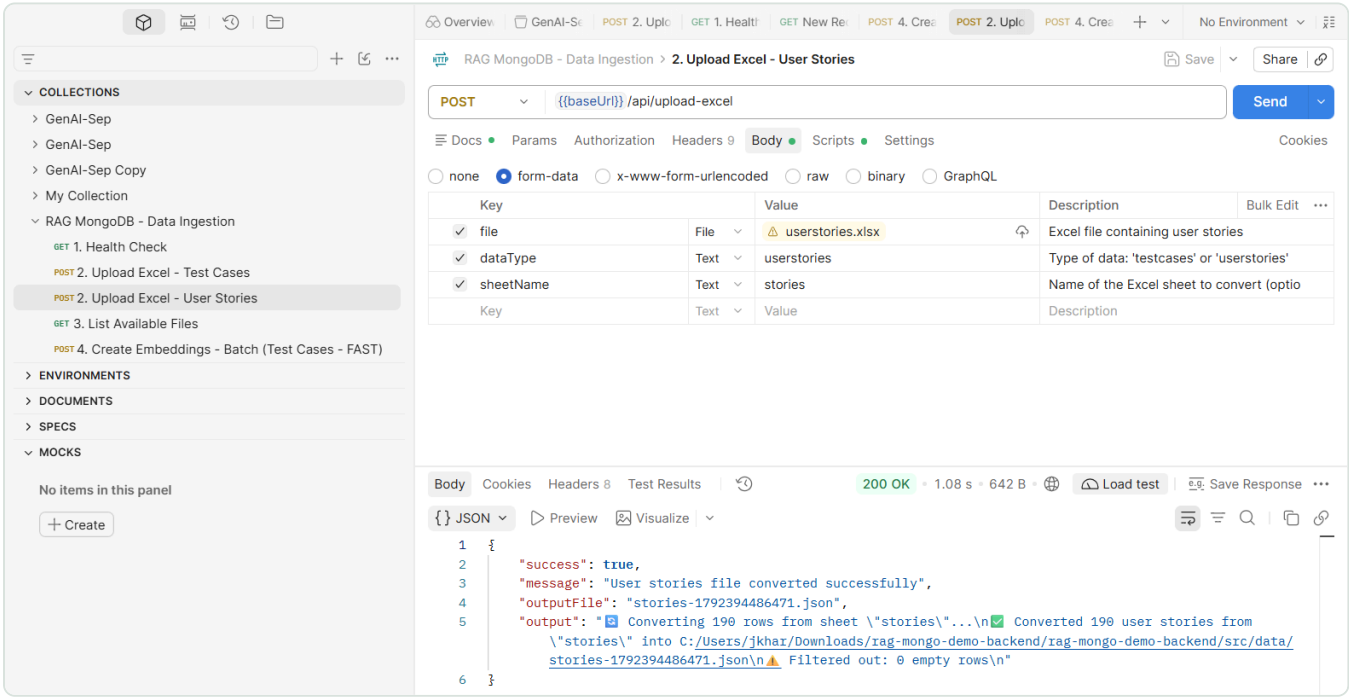


Figure 4. 190 user stories converted to `stories-1792394486471.json`, with 0 empty rows filtered out.

5 Generate embeddings and insert into MongoDB

The batch script `create-userstories-embeddings-batch-mistral.js` embeds each story with `mistral-embed` and inserts the documents into `Janani_Collection2`.

```
PS C:\Users\jkhar\Downloads\rag-mongo-demo-backend\rag-mongo-demo-backend> node .\src\scripts\embeddings\create-userstories-embeddings-batch-mistral.js
   ✅ Successfully Processed: 190
   ❌ Failed: 0
   📈 Success Rate: 100.0%
   💰 Total Cost: $0.003068
   🔢 Total Tokens: 30,678
   📊 Average Cost per User Story: $0.00001615
   📊 Average Tokens per User Story: 161
   💡 Cost Savings: ~90% cheaper than OpenAI embeddings
PS C:\Users\jkhar\Downloads\rag-mongo-demo-backend\rag-mongo-demo-backend>
```

Figure 5. 190 processed, 0 failed, 100% success. 30,678 tokens in total, $0.003068 total cost (about $0.000016 per story).

6 Verify the data in MongoDB Atlas

Data Explorer shows `Cluster0 › JananiKhartik › Janani_Collection2` with **190 documents** and 1 index, which matches the source spreadsheet exactly.

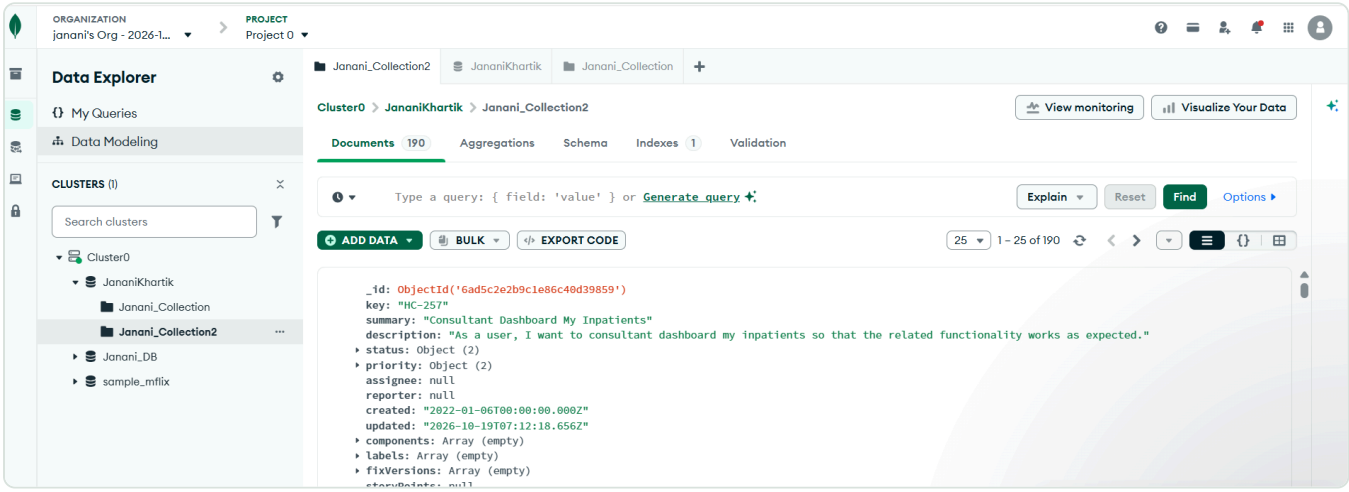


Figure 6. Document HC-257, "Consultant Dashboard My Inpatients", stored in Atlas.

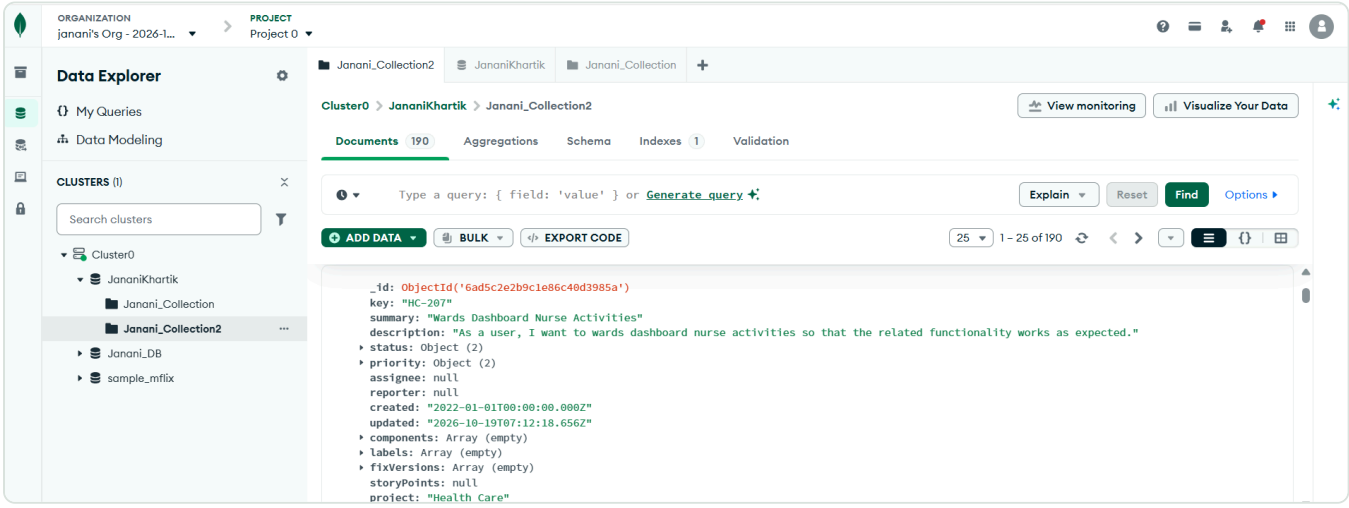


Figure 7. Document HC-207, "Wards Dashboard Nurse Activities", from the same collection.

Conclusion

- All 190 user stories moved from Excel to MongoDB Atlas with no data loss. The count matches at every stage: 190 rows, 190 JSON records, 190 embeddings and 190 documents.
- Mistral's `mistral-embed` embedded the full dataset for about $0.003. The script reports this as roughly 90% cheaper than OpenAI embeddings.
- User stories and test cases live in separate collections, each with its own vector index, so either one can be queried for retrieval-augmented generation.
- The Postman collection makes the ingestion steps repeatable and gives a quick way to check the API.